



CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342 - G5

System Integration and Architecture

System Design Document (SDD)

Project Title: Event Wear Lease System

Prepared By: Palicte, Jasmine Ciely Napoles

Version: 2

Date: May 27, 2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	02/20/26	Palicte, Jasmine Ciely	Initial draft	Draft
0.2	02/27/26	Palicte, Jasmine Ciely	Initial draft	Draft
0.3	03/31/26	Palicte, Jasmine Ciely	Added API specifications	Revised
0.4	04/07/26	Palicte, Jasmine Ciely	Updated database design	Revised
0.5	04/12/26	Palicte, Jasmine Ciely	Added/Updated UI/UX designs	Revised
0.6	04/13/26	Palicte, Jasmine Ciely	Revised Section 1.3 — removed staff management from scope; corrected technology stack (React 19, Spring Boot 4.0.3, Java 21, Kotlin ViewBinding); updated ERD with missing fields across multiple tables	Revised
0.7	05/09/206	Palicte, Jasmine Ciely	Revised Section 2.0 — added missing features (admin-created bookings, customer date editing, payment marking, activate/deactivate customer accounts); updated Section 5.0 with complete API endpoint list	Revised
1	05/27/26	Palicte, Jasmine Ciely	Final review and corrections — complete document review, cross-section consistency check, formatting finalization	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE.....	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints.....	11
User Registration.....	11
User Login.....	11
6.0 DATABASE DESIGN.....	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

EventWear Lease System (Eventwearify) is a specialized event wear rental platform that enables customers to browse, schedule fittings, and lease formal or event outfits. The system supports two distinct booking flows — a **Fitting Booking** (scheduled try-on appointment) and a **Direct Booking** (immediate lease without a prior fitting) — along with a full administrative backend for managing inventory, staff, promotions, and the end-to-end booking lifecycle. The platform is built on a three-tier architecture using a Spring Boot backend API, a React web application, and a native Kotlin Android mobile app, all communicating through RESTful APIs with real-time updates delivered via Server-Sent Events (SSE).

1.2 Objectives

1. Develop a fully functional event wear lease platform supporting fitting appointments, direct bookings, lease lifecycle management, and booking-to-lease conversion.
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Kotlin Android (mobile).
3. Create RESTful APIs for communication between all system components, secured with JWT authentication and Google OAuth2.
4. Design a responsive web interface and a native Android application that provide consistent experiences for both customers and administrators.
5. Integrate real-time booking updates via SSE and cloud media storage via Supabase for inventory images and profile photos.
6. Deploy all system components to production-ready environments.

1.3 Scope

Included Features:

- User registration and authentication (email/password and Google OAuth2)
- Two booking flows: Fitting Booking (scheduled try-on) and Direct Booking (direct lease)
- Fitting-to-Lease conversion workflow with admin-controlled approval
- Booking lifecycle management (Pending → Confirmed → Picked Up → Returned / Cancelled)
- Lease extension and return processing

- Event wear inventory catalog with image/video media (Supabase CDN), search, and category filtering
- Promotions and discount management with per-item applicability
- Admin dashboard with booking pipeline metrics (fitting and direct booking stages)
- Inventory settings (base pricing, weekly and monthly discount rates)
- Booking settings (working days, time slots, slot capacity, time restrictions)
- Customer management with admin visibility into all registered users
- Profile management with photo upload (Supabase storage)
- Email notifications for booking status changes
- Real-time inventory and booking updates via SSE with auto-reconnect
- Password reset workflow (forgot password)
- Session management and JWT token blacklisting on logout
- Native Android mobile app for both customers and administrators
- PostgreSQL database with a relational schema

Excluded Features:

- Real payment gateway integration
- Physical delivery or shipping management
- Automated inventory stock-level tracking
- Push notifications (mobile)
- Advanced analytics dashboard (booking trends, revenue charts)
- Multi-branch or multi-location support
- Social media integrations beyond Google OAuth2
- Product reviews and ratings

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the EventWear Lease System (Eventwearify). It provides detailed requirements, architectural decisions, API contracts, database design, and implementation roadmap to guide development and ensure all components — the Spring Boot backend, React web application, and Kotlin Android mobile app — integrate seamlessly. The document is intended to serve as the authoritative reference for the system's structure and behavior, covering both the

customer-facing rental experience and the administrative management of inventory, bookings, and promotions.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: EventWear Lease System (Eventwearify)

Domain: Event Wear Rental / Lease Management

Primary Users:

1. Customers
2. Administrators

Problem Statement: Event wear rental businesses face significant challenges managing fitting appointments, direct leases, inventory availability, and the full booking lifecycle through manual or fragmented processes, leading to scheduling conflicts, poor customer experience, and administrative overhead.

Solution: A digital event wear lease management platform that streamlines both fitting appointment scheduling and direct lease bookings, with real-time inventory tracking, automated booking lifecycle management, and a consistent experience across web and mobile platforms for both customers and administrators.

2.2 Core User Journeys

Journey 1: Public Visitor — Discovery and Sign-Up

1. Visitor lands on the public homepage
2. Views the Hero section introducing the platform
3. Reads the "How It Works" section (Browse, Select Date, Receive and Wear, Return)
4. Clicks the Call to Action and is directed to the registration page
5. Creates an account and is redirected to the customer dashboard

Journey 2: Customer — Fitting Booking

1. Customer logs in via email/password or Google OAuth2
2. Views the customer dashboard showing active, completed, and cancelled booking summaries
3. Navigates to Browse Outfits and searches or filters the catalog by category

4. Views item detail with sizes, pricing, and active promotions
5. Selects "Book Fitting" and chooses a date, available time slot, preferred size, and optional notes
6. Submits the booking and receives an email confirmation
7. Views and manages the fitting booking under "My Bookings"

Journey 3: Customer — Direct Booking

1. Customer logs in and browses the event wear catalog
2. Selects an item and clicks "Book Direct"
3. Chooses a lease start and end date, with unavailable dates blocked on the calendar
4. Reviews the price preview including applicable weekly or monthly discounts
5. Submits the direct booking request and receives an email confirmation
6. Views the booking under "My Bookings" and may edit the dates while still pending

Journey 4: Administrator — Booking Management

1. Admin logs in and views the dashboard with fitting and direct booking pipeline metrics
2. Reviews pending fitting and direct booking lists
3. Confirms, reschedules, or cancels bookings as needed
4. Optionally creates a booking on behalf of a walk-in or phone customer
5. Converts a confirmed fitting appointment into a direct lease (Fitting-to-Lease)
6. Marks direct bookings as paid, picked up, extended, or returned
7. Item availability updates automatically after lease return

Journey 5: Administrator — Inventory Management

1. Admin navigates to the Inventory section
2. Adds new event wear items with images or video uploaded to Supabase CDN

3. Edits item details including name, size, price, category, and status
4. Manually marks an item as Available when needed
5. Creates and manages promotions with per-item applicability
6. Configures inventory settings such as pricing and discount rates
7. Filters inventory by status: Available, Maintenance, or Pending Review

2.3 Feature List (MoSCoW)

MUST HAVE

1. Public landing page with Hero, How It Works, and Call to Action sections
2. User authentication (register, login, logout, Google OAuth2)
3. Event wear inventory catalog (list, detail, search, category filter)
4. Fitting Booking flow (date, time slot, preferred size, notes, slot capacity validation)
5. Direct Booking flow (lease dates, price preview, availability check)
6. Admin panel (inventory CRUD, booking management, customer management)

SHOULD HAVE

1. Fitting-to-Lease conversion workflow
2. Booking lifecycle management with email notifications on status changes
3. Admin-created bookings on behalf of customers (walk-in or phone bookings)
4. Customer editing of pending direct booking dates
5. Admin payment marking for direct bookings
6. Promotions and discount management with per-item applicability
7. Booking settings (working days, time slots, restrictions) and inventory settings (pricing, discounts)
8. Customer profile management with photo upload
9. Admin customer account management (view, search, activate/deactivate)
10. Real-time inventory and booking updates via Server-Sent Events (SSE)

COULD HAVE

1. Lease extension and return processing

2. Customer-initiated fitting rescheduling
3. Undo pickup for direct bookings
4. Manual mark item available (admin override)
5. Password reset via OTP email
6. JWT token blacklisting on logout

WON'T HAVE

1. Real payment gateway integration
2. Push notifications (mobile)
3. Automated inventory stock-level tracking
4. Advanced analytics and reporting dashboard
5. Multi-branch or multi-location support
6. Product reviews and ratings

2.4 Detailed Feature Specifications

Feature: Public Landing Page

- **Screens:** Home (public, no login required)
- **Sections:** Hero (platform introduction and CTA), How It Works (4-step rental process: Browse, Select Date, Receive and Wear, Return Easily), Call to Action (directs to registration)
- **Access:** Unauthenticated visitors; redirects logged-in users to their respective dashboard

Feature: User Authentication

- **Screens:** Registration, Login, Forgot Password (OTP), Change Password
- **Fields:** First Name, Last Name, Email, Password, Phone
- **Validation:** Email format, password strength, field uniqueness
- **API Endpoints:** POST /api/auth/register, POST /api/auth/login, POST /api/auth/logout, POST /api/auth/forgot-password, POST /api/auth/verify-otp, POST /api/auth/reset-password, POST /api/auth/change-password, POST /api/auth/validate-token
- **Security:** JWT tokens, bcrypt password hashing, token blacklisting on logout, Google OAuth2

Feature: Customer Dashboard Overview

- **Screens:** Customer Dashboard — Home tab
- **Display:** Booking activity summary (active, completed, cancelled counts), recent fitting and direct booking cards with status badges
- **Real-time:** Refreshes via SSE on booking status changes

Feature: Inventory Catalog

- **Screens:** Browse Outfits (web/mobile), Item Detail
- **Display:** Grid or list layout with item images/video, names, prices, and active promotions; toggle between grid and list view
- **Search:** By item name; filter by category and availability status
- **API Endpoints:** GET /api/inventory/items, GET /api/inventory/items/{id}
- **Admin Functions:** POST /api/admin/inventory/items, PUT /api/admin/inventory/items/{id}, DELETE /api/admin/inventory/items/{id}, PUT /api/admin/inventory/items/{id}/mark-available
- **Media:** Image and video upload to Supabase CDN

Feature: Fitting Booking

- **Screens:** Book Fitting Dialog, My Bookings — Fitting tab
- **Fields:** Item, Date, Time Slot, Preferred Size, Notes
- **Validation:** Working-day check, time restrictions, slot capacity (max 5 per slot), per-item date uniqueness
- **API Endpoints:** POST /api/inventory/book-fitting, GET /api/inventory/bookings/my, PATCH /api/inventory/bookings/{bookingId}/cancel, PATCH /api/inventory/bookings/{bookingId}/reschedule, GET /api/inventory/fitting/available-slots, GET /api/inventory/fitting/booked-slots, GET /api/inventory/fitting/availability
- **Email:** Confirmation email sent on successful booking

Feature: Direct Booking

- **Screens:** Book Direct Dialog, My Bookings — Direct tab
- **Fields:** Item, Lease Start Date, Lease End Date, Customer Info (auto-populated from session)
- **Validation:** Availability check against occupied date ranges, date logic enforcement, minimum lease duration
- **Price Logic:** Base price × days; weekly discount applies if days ≥ 7; monthly discount cap applies if days ≥ 30

- **Customer Self-Service:** Customers may edit start/end dates of a pending direct booking
- **API Endpoints:** POST /api/direct-bookings, GET /api/direct-bookings/my-all-bookings, PATCH /api/direct-bookings/{bookingId}/cancel, PUT /api/direct-bookings/{bookingId}/edit-dates, GET /api/direct-bookings/availability, GET /api/direct-bookings/unavailable-dates
- **Email:** Confirmation email sent on successful booking

Feature: Admin Booking Management

- **Screens:** Bookings — Fitting tab and Direct tab, Admin Dashboard pipeline metrics
- **Functions:** Create bookings on behalf of customers, update booking status, reschedule fitting, Fitting-to-Lease conversion, mark as paid, mark picked up, undo pickup, return and complete lease, extend lease, update lease dates, resend confirmation email
- **API Endpoints:**
 - Fitting: POST /api/admin/bookings/fitting, GET /api/admin/bookings/fitting, PUT /api/admin/bookings/fitting/{id}/status, PUT /api/admin/bookings/fitting/{id}/lease-started, PUT /api/admin/bookings/fitting/{id}/complete-no-lease, PUT /api/admin/bookings/fitting/{id}/reschedule, POST /api/admin/bookings/fitting/{id}/resend-email
 - Direct: POST /api/admin/bookings/direct, GET /api/admin/bookings/direct, PUT /api/admin/bookings/direct/{id}/status, PUT /api/admin/bookings/direct/{id}/pickup, PUT /api/admin/bookings/direct/{id}/undo-pickup, PUT /api/direct-bookings/{id}/return, PUT /api/direct-bookings/{id}/extend, PUT /api/admin/bookings/direct/{id}/dates, PUT /api/direct-bookings/{id}/payment, POST /api/admin/bookings/direct/{id}/resend-email
- **Access Control:** ADMIN role required for all admin endpoints

Feature: Customer Management

- **Screens:** Customers — Admin panel

- **Functions:** View paginated customer list with search and status filter, view customer detail, activate or deactivate customer accounts
- **API Endpoints:** GET /api/admin/users, GET /api/admin/users/{id}, PATCH /api/admin/users/{id}/status
- **Access Control:** ADMIN role required

Feature: Profile Management

- **Screens:** Manage Profile, Change Password
- **Fields:** First Name, Last Name, Email, Phone, Profile Photo
- **API Endpoints:** GET /api/user/profile, PUT /api/user/profile, POST /api/user/profile/photo, POST /api/auth/change-password
- **Media:** Profile photo upload to Supabase (JPEG, PNG, WEBP, GIF; max 5 MB)

Feature: Promotions and Discounts

- **Screens:** Promotions — Admin; Browse Outfits — Customer (promotional price displayed on items)
- **Functions:** Create, edit, and delete promotions; assign promotions to specific inventory items by ID; supports percentage-off and fixed-amount-off discount types
- **API Endpoints:** GET /api/inventory/promotions, POST /api/admin/promotions, PUT /api/admin/promotions/{id}, DELETE /api/admin/promotions/{id}

Feature: Booking and Inventory Settings

- **Screens:** Settings — Admin
- **Booking Settings:** Working days, available time slots, fitting duration, slot capacity, time restriction toggle
- **Inventory Settings:** Base pricing, minimum lease days, weekly discount rate, monthly discount cap
- **API Endpoints:** GET /api/admin/booking-settings, PUT /api/admin/booking-settings, GET /api/admin/inventory-settings, PUT /api/admin/inventory-settings

2.5 Acceptance Criteria

AC-1: Successful Customer Registration

- Given I am a new visitor on the landing page
- When I click the Call to Action and enter valid first name, last name, email, phone, and strong password
- And click "Create Account"
- Then my account should be created with a CUSTOMER role
- And I should be automatically logged in and redirected to the customer dashboard

AC-2: Fitting Booking Flow

- Given I am logged in as a customer
- When I browse the catalog, select an item, and click "Book Fitting"
- And I choose a valid date, an available time slot, preferred size, and optional notes
- And the selected slot has not reached the maximum capacity of 5
- Then a fitting booking should be created with status PENDING
- And I should receive a booking confirmation email
- And the booking should appear under "My Bookings"

AC-3: Direct Booking Flow

- Given I am logged in as a customer
- When I browse the catalog, select an item, and click "Book Direct"
- And I select valid start and end dates that do not overlap with existing bookings
- Then a direct booking should be created with status PENDING
- And the price preview should reflect any applicable weekly or monthly discounts
- And I should receive a booking confirmation email

AC-4: Customer Editing of Pending Direct Booking

- Given I am logged in as a customer
- When I view a direct booking that is still in PENDING status
- And I update the start and end dates to new available dates
- Then the booking dates should be updated without changing the booking status
- And the updated dates should be reflected immediately in "My Bookings"

AC-5: Admin Fitting-to-Lease Conversion

- Given I am logged in as an administrator
- When a fitting booking is confirmed and the customer decides to proceed with a lease
- And I click "Convert to Lease" and provide the lease start and end dates
- Then the fitting booking status should update to LEASE_STARTED

- And a corresponding direct booking record should be automatically created
- And the item's availability calendar should reflect the new lease period

AC-6: Admin Payment Marking

- Given I am logged in as an administrator
- When I select a confirmed direct booking and mark it as paid
- Then the booking's payment status should be updated
- And the change should be broadcast to all connected clients via SSE

AC-7: Admin Customer Account Management

- Given I am logged in as an administrator
- When I navigate to the Customers section and deactivate a customer account
- Then that customer should be unable to log in until the account is reactivated
- And the customer's status should reflect as inactive in the customer list

AC-8: Admin Inventory Management

- Given I am logged in as an administrator
- When I add a new event wear item with valid details and at least one image
- And save the item
- Then the item should appear in the customer-facing catalog
- And be available for both fitting and direct bookings

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests under normal load
- Web page load time: ≤ 3 seconds on a broadband connection
- Mobile app cold start: ≤ 3 seconds on Android API Level 24 and above
- Support a minimum of 100 concurrent users without degraded response times
- Database queries complete within 500ms for standard read and write operations
- Inventory catalog and booking list pages must load and render within 2 seconds including media thumbnails served from Supabase CDN
- SSE connections must reconnect automatically within 5 seconds of a dropped connection to maintain real-time booking update delivery
- Profile photo and inventory media uploads (images and video) must complete within 10 seconds for files up to 5 MB on a standard broadband connection

3.2 Security Requirements

- HTTPS enforced for all client-server communications across web and mobile

- JWT token-based authentication for all protected API endpoints; tokens are validated on every request
- Tokens are blacklisted server-side on logout to prevent reuse of invalidated sessions
- Password hashing with bcrypt (minimum 10 salt rounds)
- Google OAuth2 integration handled server-side; OAuth error codes on the frontend are whitelisted to prevent information leakage
- SQL injection prevention enforced via Spring Data JPA and Hibernate parameterized queries; no raw SQL string concatenation
- XSS protection enforced by returning pre-defined error messages on the frontend instead of echoing raw server responses
- CORS policy configured to restrict cross-origin requests to approved origins only
- Input validation enforced at the API layer using Jakarta Bean Validation annotations on all request DTOs
- Admin endpoints restricted by role using Spring Security `@PreAuthorize("hasRole('ADMIN')")` on all admin controllers
- Customer endpoints restricted so a customer can only access their own bookings and profile; cross-user access returns 403 Forbidden
- Philippine phone number format validated on input (must match +63XXXXXXXXXX)
- Profile photo uploads restricted to JPEG, PNG, WEBP, and GIF formats with a maximum file size of 5 MB
- Session expiry detected client-side via token validation on page load; expired sessions redirect to the login page with a session timeout notification

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, and Edge (latest 2 major versions)
- **Android:** API Level 24 and above (Android 7.0 Nougat and higher); minimum SDK confirmed at 24
- Screen Sizes:
 - Mobile: 360px and above
 - Tablet: 768px and above
 - Desktop: 1024px and above

- **Responsive Design:** The web application must adapt layout and navigation for all three screen size ranges; the admin sidebar supports a collapsible mode for smaller screens
- **Backend Runtime:** Java 17+, Spring Boot 3.x
- **Database:** PostgreSQL (latest stable release)
- **Media Storage:** Supabase CDN for inventory images, inventory videos, and user profile photos
- **Development OS:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

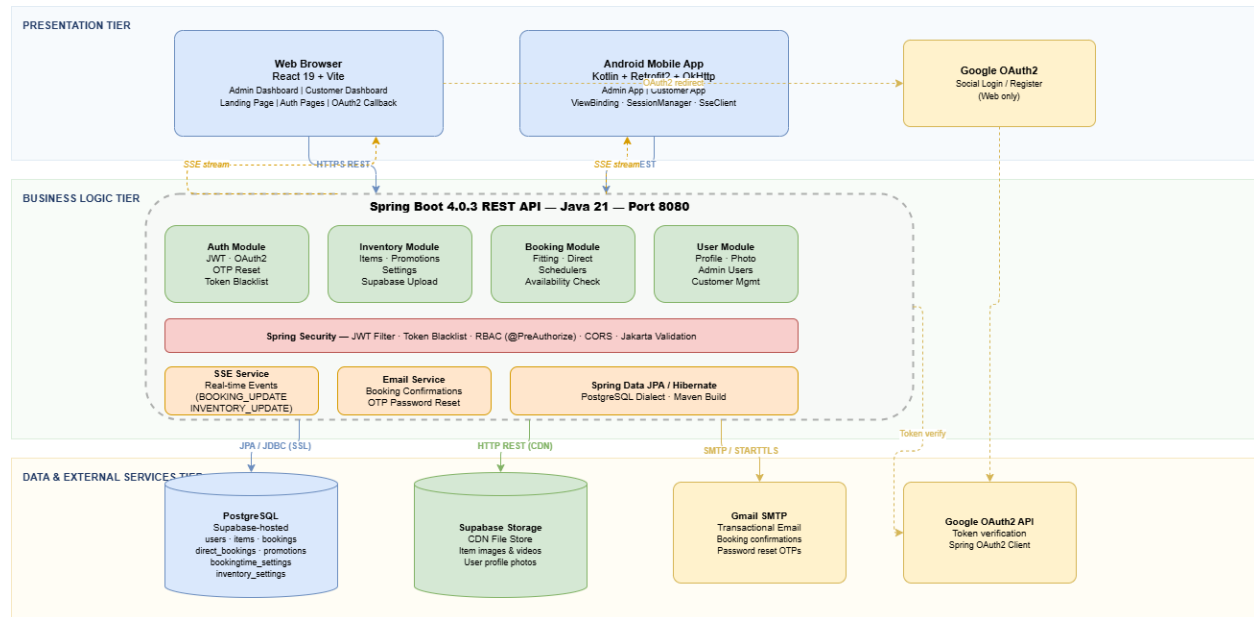
3.4 Usability Requirements

- A new customer must be able to complete a fitting or direct booking within 5 minutes of registering, including browsing the catalog and submitting the booking form
- Administrators must be able to create a booking on behalf of a walk-in customer within 3 minutes using the admin booking creation modal
- Navigation must be consistent across all pages; the customer dashboard uses a persistent sidebar with four tabs (Dashboard, Browse Outfits, My Profile, My Bookings), and the admin dashboard uses a persistent sidebar with five tabs (Dashboard, Inventory, Bookings, Customers, Profile)
- All status changes to bookings and inventory must be reflected in real time across all connected sessions via SSE without requiring a manual page refresh
- Error messages must be clear, specific, and actionable — indicating what went wrong and how the user can resolve it (e.g., slot fully booked, dates unavailable, invalid phone format)
- Booking date pickers must visually block unavailable dates and working-day restrictions to prevent invalid selections before form submission
- Touch targets on mobile must be a minimum of 44×44px
- The Android mobile app must support both customer and administrator roles with distinct navigation flows, routing determined automatically by the role returned at login
- All form fields must include appropriate placeholder text and inline validation feedback
- The admin booking pipeline on the dashboard must surface at-a-glance counts for each booking stage (Pending, Confirmed, Active Lease, Returned) for both fitting and direct bookings

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 21, Spring Boot 4.0.3, Spring Security, Spring Data JPA / Hibernate
- **Database:** PostgreSQL (Supabase-hosted)
- **Web Frontend:** React 19, JavaScript (JSX), Vite 7, Axios, React Router DOM, Lucide React
- **Mobile:** Kotlin, Retrofit2, OkHttp3, Gson, Coroutines, ViewBinding, Material 3
- **Build Tools:** Maven (Backend), npm + Vite (Web), Gradle Kotlin DSL (Android)
- **External Services:** Supabase Storage CDN, Gmail SMTP, Google OAuth2
- **Real-time:** Server-Sent Events (SSE) with auto-reconnect
- **Mobile Distribution:** APK (Android minSdk 24 / targetSdk 34)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests and responses
Authentication	Authorization: Bearer <jwt-token> header on all protected endpoints
Date Format	ISO 8601 — yyyy-MM-dd for dates, yyyy-MM-ddTHH:mm:ss for timestamps

Success Response — each endpoint returns its own data object directly (no shared wrapper envelope):

```
{  
  "field1": "value",  
  "field2": "value"  
}
```

Error Response — all errors follow the ErrorResponse structure produced by GlobalExceptionHandler:

```
{  
  "timestamp": "2026-06-04T10:30:00",  
  "status": 401,  
  "error": "Unauthorized",  
  "message": "Invalid email or password",  
  "path": "/api/auth/login",  
  "validationErrors": null  
}
```

For validation errors, validationErrors is a field-keyed map instead of null:

```
{  
  "timestamp": "2026-06-04T10:30:00",  
  "status": 400,  
  "error": "Validation Failed",  
  "message": "Input validation failed",  
  "path": "/api/auth/register",  
  "validationErrors": {  
    "firstName": "First name is required",  
    "password": "Password must contain at least 8 characters, one uppercase, one lowercase, one number, and one special character"  
  }  
}
```

}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Register a new customer account
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "firstName": "Jane", "lastName": "Cruz", "email": "jane@example.com", "password": "Secure@123" }</pre>
Response Structure	<pre>{ "token": "<jwt-token>", "email": "jane@example.com", "firstName": "Jane", "lastName": "Cruz", "role": "CUSTOMER", "mustChangePassword": false, "message": "Registration successful", "status": 201 }</pre>

User Login

Description	Authenticate a user and return a JWT token
API URL	/api/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": "jane@example.com", "password": "Secure@123" }</pre>
Response Structure	<pre>{ "token": "<jwt-token>", "email": "jane@example.com",</pre>

	<pre> "firstName": "Jane", "lastName": "Cruz", "role": "CUSTOMER", "mustChangePassword": false, "message": "Login successful", "status": 200 } </pre>
--	---

User Logout

Description	Blacklist the current JWT token to invalidate the session
API URL	/api/auth/logout
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	
Response Structure	<pre> { "message": "Logged out successfully" } </pre>

Change Password

Description	Change the authenticated user's password
API URL	/api/auth/change-password
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	<pre> { "currentPassword": "OldPass@1", "newPassword": "NewPass@2" } </pre>
Response Structure	<pre> { "message": "Password changed successfully" } </pre>

Forgot Password — Send OTP

Description	Send a one-time password to the user's email for password reset
API URL	/api/auth/forgot-password
HTTP Method	POST

Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": "jane@example.com" }
Response Structure	{ "message": "If that email is registered, an OTP has been sent." }

Forgot Password — Verify OTP

API URL	/api/auth/verify-otp
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": "jane@example.com", "otp": "482910" }
Response Structure	{ "message": "OTP verified successfully." }

Forgot Password — Reset Password

API URL	/api/auth/reset-password
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": "jane@example.com", "otp": "482910", "newPassword": "NewPass@2" }
Response Structure	{ "message": "Password reset successfully." }

Fitting Booking Endpoints

Fitting Booking Endpoints

Description	Book a fitting appointment for an inventory item
API URL	/api/inventory/book-fitting
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token — CUSTOMER role
Request Payload	<pre>{ "itemId": "item-uuid", "itemName": "Burgundy Barong", "fittingDate": "2026-06-20", "fittingTime": "10:00 AM", "customerName": "Jane Cruz", "customerEmail": "jane@example.com", "customerPhone": "+639123456789", "preferredSize": "M", "notes": "Please prepare alterations.", "userId": 1 }</pre>
Response Structure	<pre>{ "bookingId": "FT1749042001234", "status": "PENDING", "message": "Booking confirmed" }</pre>

Get My Fitting Bookings (Customer)

Description	Book a fitting appointment for an inventory item
API URL	/api/inventory/bookings/my
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token — CUSTOMER role
Request Payload	
Response Structure	<pre>[{ "id": "uuid", "bookingId": "FT1749042001234", "itemId": "item-uuid", "itemName": "Burgundy Barong", "fittingDate": "2026-06-20", "fittingTime": "10:00 AM",</pre>

	<pre> "customerName": "Jane Cruz", "customerEmail": "jane@example.com", "customerPhone": "+639123456789", "preferredSize": "M", "notes": "Please prepare alterations.", "userId": 1, "status": "PENDING", "createdAt": "2026-06-04T10:00:00", "updatedAt": "2026-06-04T10:00:00" }]</pre>
--	--

Cancel Fitting Booking (Customer)

Description	Book a fitting appointment for an inventory item
API URL	/api/inventory/bookings/{bookingId}/cancel
HTTP Method	PATCH
Format	JSON for all requests/responses
Authentication	Bearer token — CUSTOMER role
Request Payload	
Response Structure	<pre> { "message": "Booking cancelled successfully" }</pre>

Check Fitting Slot Availability

Description	Book a fitting appointment for an inventory item
API URL	/api/inventory/fitting/availability?date=2026-06-20&time=10:00 AM
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token — CUSTOMER role
Request Payload	
Response Structure	["9:00 AM", "9:30 AM", "10:00 AM", "10:30 AM"]

Direct Booking Endpoints

Create Direct Booking (Customer)

Description	Create a direct lease booking for an inventory item
API URL	/api/direct-bookings
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token — CUSTOMER role
Request Payload	<pre>{ "inventoryItemId": "item-uuid", "itemName": "Burgundy Barong", "startDate": "2026-06-25", "endDate": "2026-07-02", "totalDays": 7, "basePrice": 1500.00, "discountAmount": 100.00, "finalPrice": 1400.00, "notes": "Event: wedding", "customerName": "Jane Cruz", "customerEmail": "jane@example.com", "customerPhone": "+639123456789", "preferredSize": "M" }</pre>
Response Structure	<pre>{ "id": "uuid", "userId": "user-email", "inventoryItemId": "item-uuid", "itemName": "Burgundy Barong", "bookingType": "DIRECT", "startDate": "2026-06-25", "endDate": "2026-07-02", "totalDays": 7, "basePrice": 1500.00, "discountAmount": 100.00, "finalPrice": 1400.00, "bookingStatus": "Pending", "notes": "Event: wedding", "customerName": "Jane Cruz", "customerEmail": "jane@example.com", "customerPhone": "+639123456789", "preferredSize": "M", "extended": false, "paymentStatus": null, "paymentDate": null, }</pre>

	<pre>"paymentNotes": null, "createdAt": "2026-06-04T10:00:00", "updatedAt": "2026-06-04T10:00:00" }</pre>
--	---

Get My Direct Bookings (Customer)

API URL	/api/direct-bookings/my-all-bookings
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token — CUSTOMER role
Request Payload	
Response Structure	array of DirectBookingResponse objects (same structure as above).

Edit Direct Booking Dates (Customer)

Description	Update start and end dates of a pending direct booking
API URL	/api/direct-bookings/{bookingId}/edit-dates
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token — CUSTOMER role
Request Payload	<pre>{ "startDate": "2026-06-28", "endDate": "2026-07-05" }</pre>
Response Structure	updated DirectBookingResponse.

Check Direct Booking Availability

API URL	/api/direct-bookings/availability?itemId=&startDate=2026-06-25&endDate=2026-07-02
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token — CUSTOMER role
Request Payload	
Response Structure	<pre>{ "available": true }</pre>

Admin Booking Management Endpoints

Get All Fitting Bookings (Admin)

API URL	/api/admin/bookings/fitting?page=0&size=20
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	
Response Structure	paginated BookingDetailResponse list.

Update Fitting Booking Status (Admin)

API URL	/api/admin/bookings/fitting/{bookingId}/status?status=CONFIRMED
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	
Response Structure	{ "id": "uuid", "status": "CONFIRMED" }

Convert Fitting to Lease (Admin)

DESCRIPTION	Mark a fitting booking as lease-started and link it to a direct booking
API URL	/api/admin/bookings/fitting/{bookingId}/lease-started
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	{ "directBookingId": "direct-booking-uuid" }
Response Structure	{ "id": "uuid", "status": "LEASE_STARTED", "leaseStarted": true }

Mark Direct Booking Picked Up (Admin)

API URL	/api/admin/bookings/direct/{bookingId}/pickup
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	
Response Structure	updated DirectBookingResponse.

Return and Complete Lease (Admin)

API URL	/api/direct-bookings/{bookingId}/return
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	
Response Structure	updated DirectBookingResponse with bookingStatus: "Completed".

Extend Lease (Admin)

API URL	/api/direct-bookings/{bookingId}/extend
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	{ "newEndDate": "2026-07-10" }
Response Structure	updated DirectBookingResponse with extended: true.

Mark Direct Booking as Paid (Admin)

API URL	/api/direct-bookings/{bookingId}/payment
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	{ "notes": "Paid via GCash" }
Response Structure	updated DirectBookingResponse with paymentStatus populated.

User Profile Endpoints

Get Profile

API URL	/api/user/profile
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	
Response Structure	<pre>{ "id": "uuid", "firstName": "Jane", "lastName": "Cruz", "email": "jane@example.com", "phone": "+639123456789", "profilePhotoUrl": "https://cdn.supabase.co/.../avatar.jpg", "role": "CUSTOMER", "active": true, "createdAt": "2026-05-01T08:00:00", "lastLoginAt": "2026-06-04T10:00:00", "emailChanged": false }</pre>

Update Profile

API URL	/api/user/profile
HTTP Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	<pre>{ "firstName": "Jane", "lastName": "Dela Cruz", "email": "jane@example.com", "phone": "+639123456789" }</pre>
Response Structure	updated ProfileResponse.

Upload Profile Photo

API URL	/api/user/profile/photo
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token required
Content-Type	multipart/form-data

Field name	photo
Allowed types	JPEG, PNG, WEBP, GIF
Max size	5 MB
Response Structure	updated ProfileResponse with new profilePhotoUrl.

Promotions Endpoints

Get All Promotions

API URL	/api/inventory/promotions
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	
Response Structure	<pre>[{ "id": "promo-uuid", "code": "SAVE100", "type": "fixed", "value": 100.0, "start": "2026-06-01", "end": "2026-06-30", "active": true, "items": ["item-uuid-1", "item-uuid-2"] }]</pre>

Get All Promotions

API URL	/api/admin/promotions
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	<pre>{ "code": "SAVE100", "type": "fixed", "value": 100.0, "start": "2026-06-01", "end": "2026-06-30", "active": true, "items": ["item-uuid-1", "item-uuid-2"] }</pre>
Response Structure	created PromotionResponse.

Settings Endpoints

Get Booking Settings (Admin)

API URL	/api/admin/booking-settings
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	
Response Structure	<pre>{ "enableTimeRestrictions": true, "shopOpenTime": "09:00", "shopCloseTime": "17:00", "workingDays": [1, 2, 3, 4, 5], "timezone": "Asia/Manila", "autoApproveThreshold": 0, "fittingDurationMinutes": 30 }</pre>

Get Inventory Settings (Admin)

API URL	/api/admin/inventory-settings
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token — ADMIN role
Request Payload	
Response Structure	<pre>{ "id": 1, "minLeaseDays": 2, "weeklyDiscount": 100, "monthlyDiscountCap": 300 }</pre>

5.3 Error Handling

HTTP Status Codes

Code	Meaning	When Used
200 OK	Successful request	Standard successful response
201 Created	Resource created	Registration, item creation
400 Bad Request	Invalid input or business rule violation	Slot fully booked, dates unavailable, validation failed

401 Unauthorized	Authentication required or failed	Missing token, expired token, invalid credentials, blacklisted token
403 Forbidden	Insufficient permissions	Customer accessing admin endpoint, cross-user data access
404 Not Found	Resource does not exist	Booking not found, item not found
409 Conflict	Duplicate resource	Email already registered
500 Internal Server Error	Unhandled server error	Unexpected exceptions

Error Code Examples

Invalid credentials (401)	{ "timestamp": "2026-06-04T10:30:00", "status": 401, "error": "Unauthorized", "message": "Invalid email or password", "path": "/api/auth/login", "validationErrors": null }
Validation failure (400)	{ "timestamp": "2026-06-04T10:30:00", "status": 400, "error": "Validation Failed", "message": "Input validation failed", "path": "/api/auth/register", "validationErrors": { "firstName": "First name is required", "password": "Password must contain at least 8 characters, one uppercase, one lowercase, one number, and one special character" } }
Expired JWT (401)	{ "timestamp": "2026-06-04T10:30:00", "status": 401, "error": "Unauthorized", "message": "Session expired. Please log in again.", "path": "/api/inventory/bookings/my", "validationErrors": null }
Duplicate email (409)	{ "timestamp": "2026-06-04T10:30:00", "status": 409, }

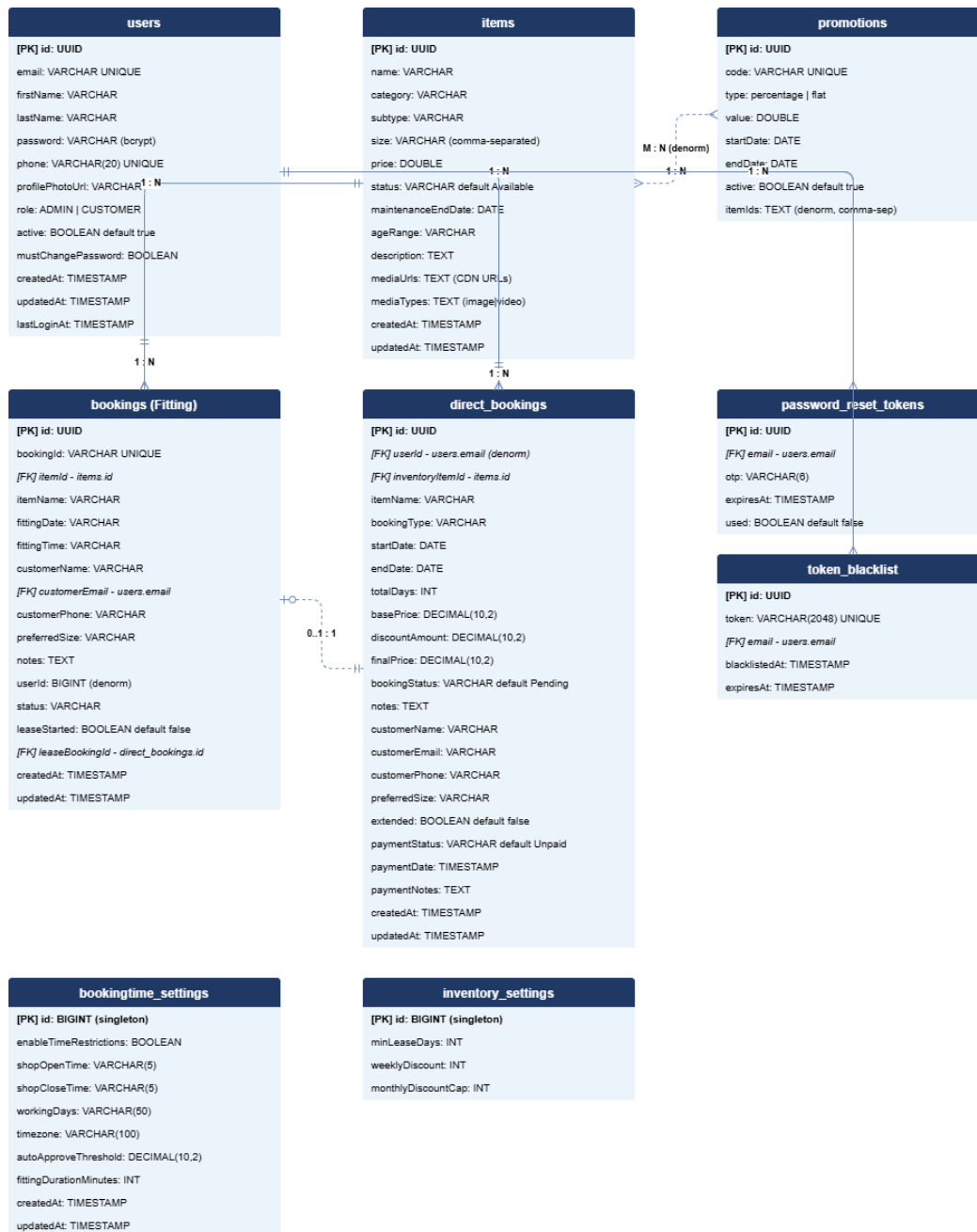
	<pre> "error": "Conflict", "message": "Email already registered", "path": "/api/auth/register", "validationErrors": null } </pre>
--	---

Common Error Codes

Scenario	Status	error	message
Wrong email or password	401	Unauthorized	Invalid email or password
JWT token expired	401	Unauthorized	Session expired. Please log in again.
Malformed or invalid JWT	401	Unauthorized	Invalid token. Please log in again.
Token blacklisted (logged out)	401	Unauthorized	Token has been revoked
Admin endpoint accessed by customer	403	Forbidden	Access denied
Email already registered	409	Conflict	Email already registered
Booking not found	404	Not Found	Booking not found
Fitting slot fully booked	400	Bad Request	This time slot is fully booked
Direct booking dates unavailable	400	Bad Request	Item is not available for the selected dates
Input validation failed	400	Validation Failed	Input validation failed
Unexpected server error	500	Internal Server Error	An unexpected error occurred

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- One-to-Many: users → bookings — A user can have multiple fitting bookings (linked via customerEmail)
- One-to-Many: users → direct_bookings — A user can have multiple direct lease bookings (linked via userId)
- One-to-Many: users → password_reset_tokens — A user can have multiple OTP reset requests
- One-to-Many: users → token_blacklist — A user can have multiple blacklisted tokens (one per logout)

- One-to-Many: items → bookings — An item can have multiple fitting bookings over time
- One-to-Many: items → direct_bookings — An item can have multiple direct bookings over time
- Zero-or-One-to-One: bookings → direct_bookings — A fitting booking may convert to a direct lease (Fitting-to-Lease); linked via leaseBookingId
- Many-to-Many (denormalized): items ↔ promotions — Promotions store applicable item IDs as a comma-separated itemIds TEXT column; no junction table

Key Tables:

Table	Purpose
users	Customer and admin accounts, authentication, profile
items	Event wear inventory catalog with media and pricing
bookings	Fitting appointment bookings
direct_bookings	Direct lease bookings with pricing and payment tracking
promotions	Discount promotions applied to specific inventory items
password_reset_tokens	OTP tokens for the forgot-password flow
token_blacklist	Invalidated JWT tokens post-logout
bookingtime_settings	Singleton row for fitting schedule configuration
inventory_settings	Singleton row for lease pricing and discount configuration

Table Structure Summary:

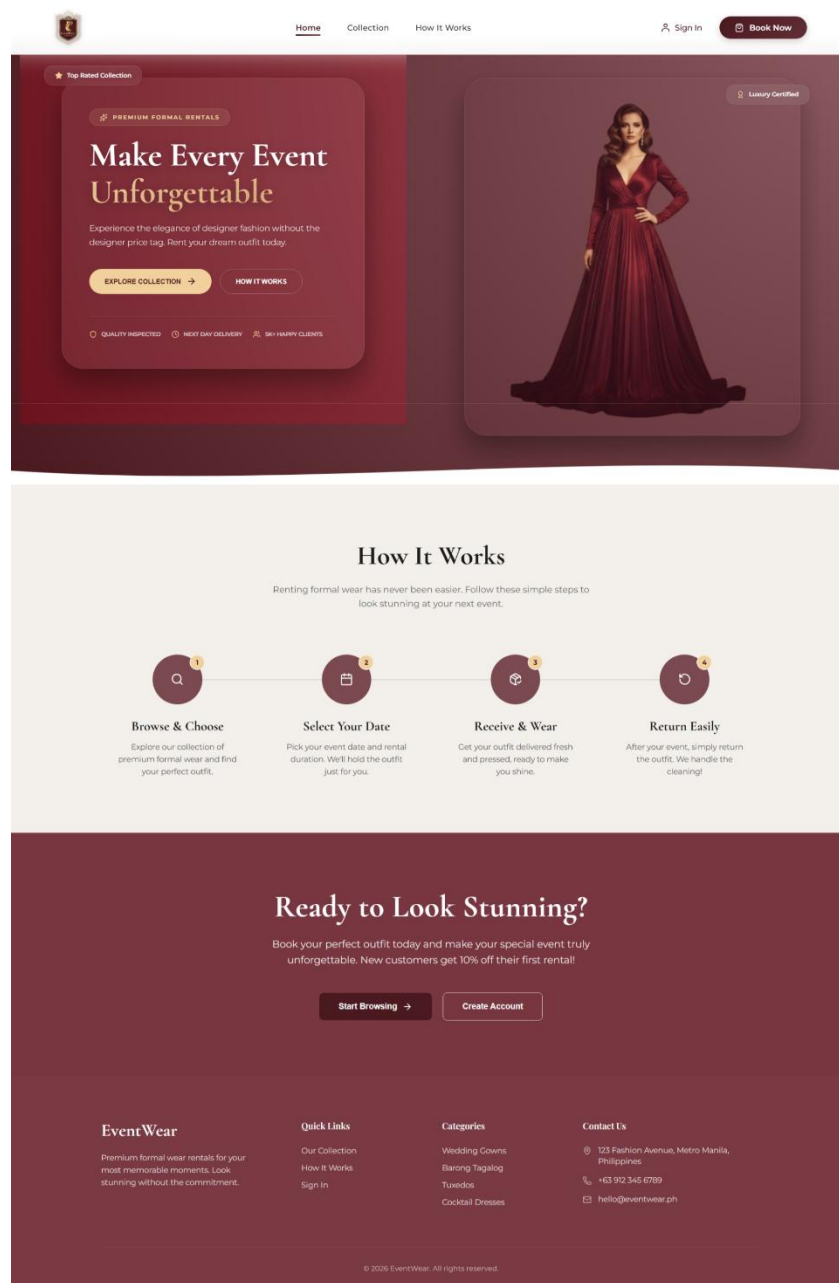
- **users:** id, email, firstName, lastName, password, phone, profilePhotoUrl, role, active, mustChangePassword, createdAt, updatedAt, lastLoginAt
- **items:** id, name, category, subtype, size, price, status, maintenanceEndDate, ageRange, description, mediaUrls, mediaTypes, createdAt, updatedAt
- **bookings:** id, bookingId, itemId, itemName, fittingDate, fittingTime, customerName, customerEmail, customerPhone, preferredSize, notes, userId, status, leaseStarted, leaseBookingId, createdAt, updatedAt
- **direct_bookings:** id, userId, inventoryItemId, itemName, bookingType, startDate, endDate, totalDays, basePrice, discountAmount, finalPrice, bookingStatus, notes, customerName, customerEmail, customerPhone, preferredSize, extended, paymentStatus, paymentDate, paymentNotes, createdAt, updatedAt
- **promotions:** id, code, type, value, startDate, endDate, active, itemIds
- **password_reset_tokens:** id, email, otp, expiresAt, used
- **token_blacklist:** id, token, email, blacklistedAt, expiresAt

- **bookingtime_settings:** id, enableTimeRestrictions, shopOpenTime, shopCloseTime, workingDays, timezone, autoApproveThreshold, fittingDurationMinutes, createdAt, updatedAt
- **inventory_settings:** id, minLeaseDays, weeklyDiscount, monthlyDiscountCap

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma



Authentication Page (Login / Register)

← BACK TO HOME



Welcome Back

Sign in to manage your bookings and rentals

SIGN IN CREATE ACCOUNT

 Sign in with Google

OR

EMAIL ADDRESS

 you@example.com

PASSWORD

[Forgot password?](#)

SIGN IN →

CLEAR FORM

Don't have an account? [Sign up here](#)

OR

Just browsing?

CONTINUE AS GUEST

← BACK TO HOME



Create Account

Join EventWear to get started with premium rentals

SIGN IN

CREATE ACCOUNT

Sign up with Google

OR

FIRST NAME

LAST NAME

Juan

Dela Cruz

EMAIL ADDRESS

you@example.com

PASSWORD

At least 8 characters

One uppercase letter

One lowercase letter

One number

One special character (!@#%&*)

CONFIRM PASSWORD

CREATE ACCOUNT →

CLEAR FORM

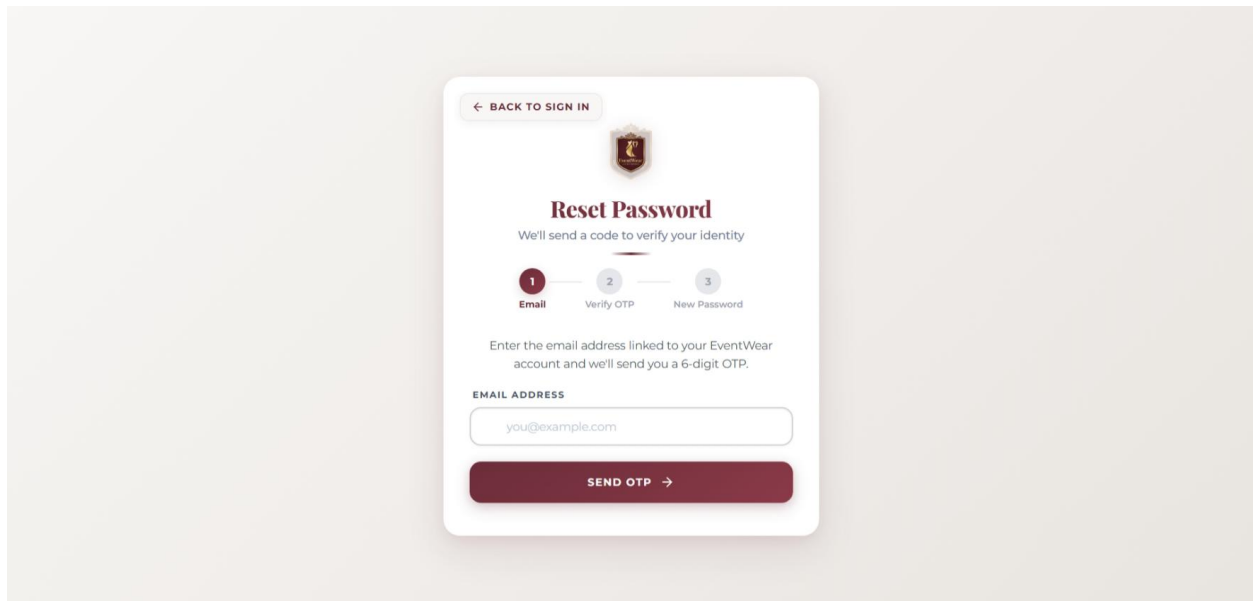
Already have an account? [Sign in here](#)

OR

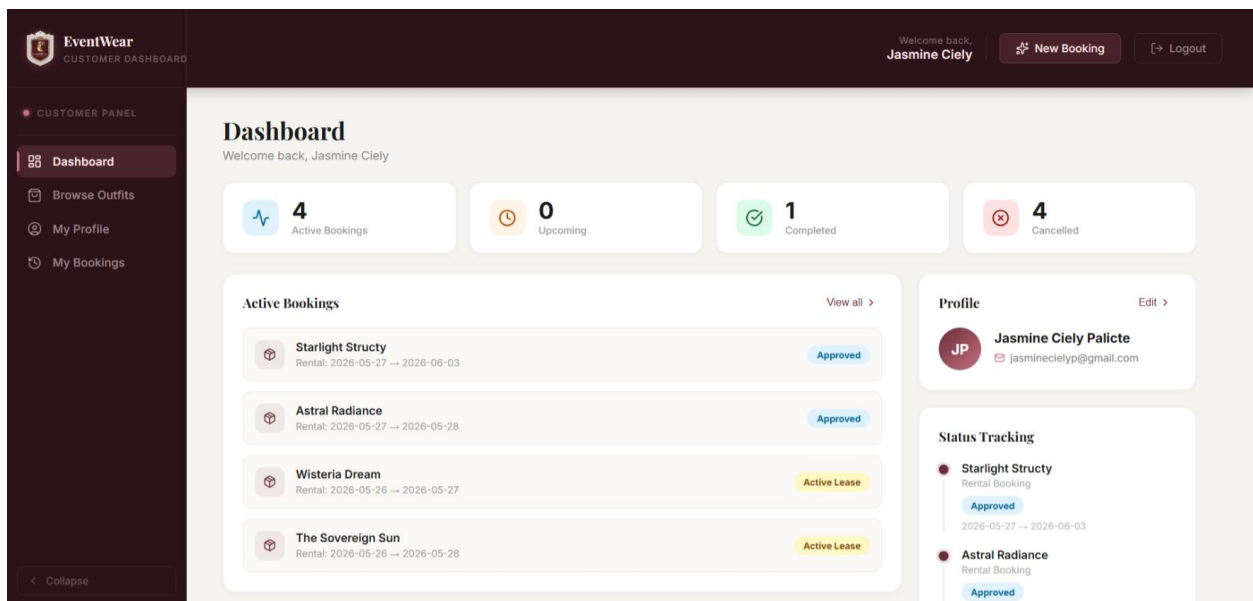
Just browsing?

CONTINUE AS GUEST

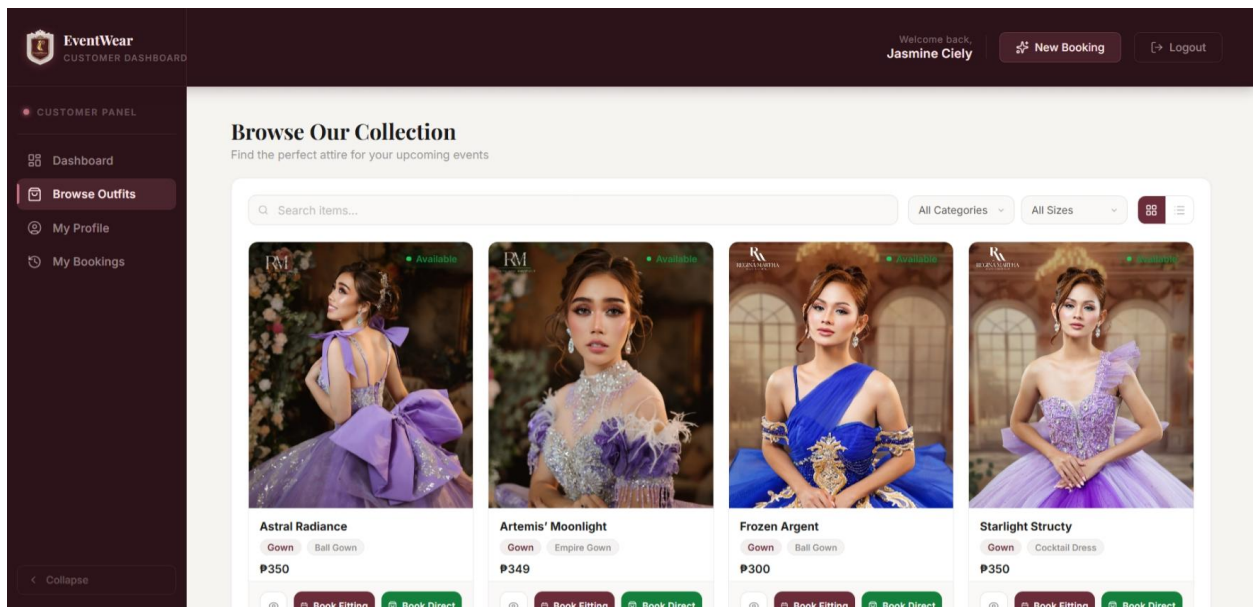
Forgot Password Page



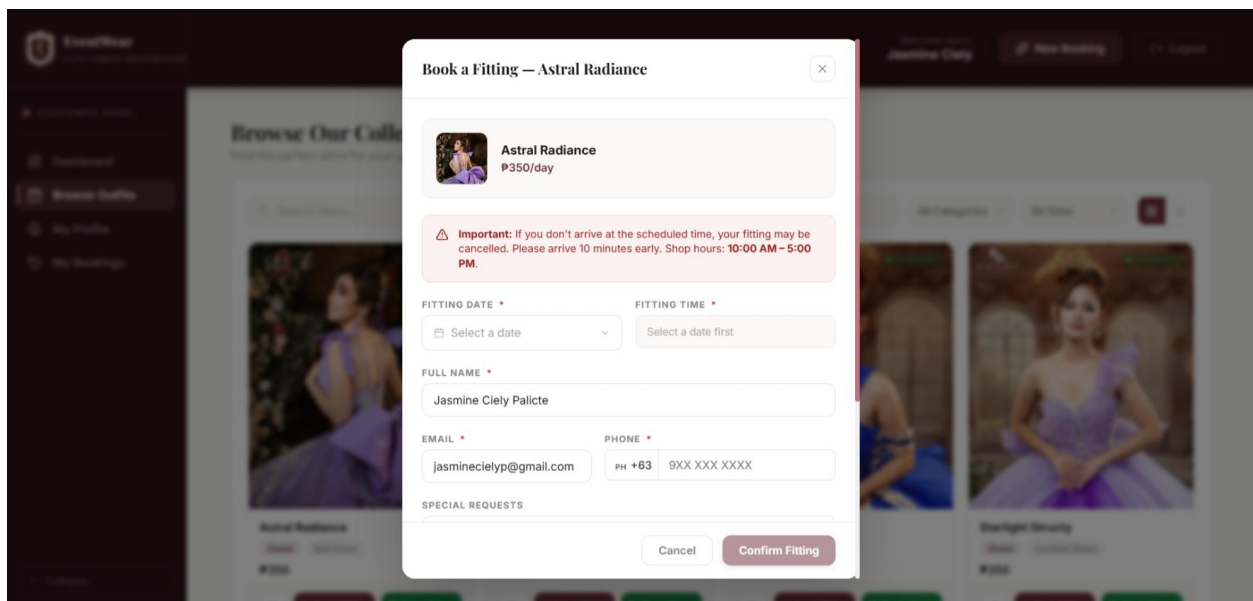
Customer Dashboard — Overview



Customer Dashboard — Browse Outfits



Customer Dashboard — Book Fitting Dialog



Customer Dashboard — Book Direct Dialog

EventWear
CUSTOMER DASHBOARD

Welcome back,
Jasmine Ciely

New Booking

Logout

CUSTOMER PANEL

Dashboard
Browse Outfits
My Profile
My Bookings

Direct Booking — Astral Radiance

Astral Radiance
P350/day

Note: Direct booking requires full payment upon confirmation. You will be contacted for payment and pickup arrangements.

RENTAL START DATE

RENTAL END DATE

Select start date

Select end date

CUSTOMER INFORMATION

FULL NAME

Jasmine Ciely Palicte

EMAIL

PHONE

jasminecielyp@gmail.com

PH +63 9XX XXX XXXX

SPECIAL INSTRUCTIONS

Cancel

Confirm Booking

Customer Dashboard — My Bookings

EventWear
CUSTOMER DASHBOARD

Welcome back,
Jasmine Ciely

New Booking

Logout

CUSTOMER PANEL

Dashboard
Browse Outfits
My Profile
My Bookings

Your Bookings

Active & Pending 2

Completed & Cancelled 0

TYPE	BOOKING ID	OUTFIT(S)	DATE / PERIOD	STATUS	ACTION	
RENTAL	46fb2266-657d-41df-9dd4-5c34537b667d	Wisteria Dream	2026-05-26 → 2026-05-27	Active Lease ✓ Paid	View Details	Edit Dates Cancel
RENTAL	1bd561b4-a868-40d8-a438-14cd7437f9c3	The Sovereign Sun	2026-05-26 → 2026-05-28	Active Lease ✓ Paid	View Details	Edit Dates Cancel

Customer Dashboard — My Profile


EventWear
CUSTOMER DASHBOARD

Welcome back,
Jasmine Ciely

[New Booking](#)

[Logout](#)

CUSTOMER PANEL

Dashboard

Browse Outfits

My Profile

My Bookings

[Collapse](#)

My Profile

Manage your account information and security settings



Jasmine Ciely Palicte
CUSTOMER
jasminecielyp@gmail.com

[Edit Profile](#)

PERSONAL INFORMATION

FIRST NAME	LAST NAME
Jasmine Ciely	Palicte
EMAIL	PHONE
jasminecielyp@gmail.com	Not set

CHANGE PASSWORD

CURRENT PASSWORD

Current Password

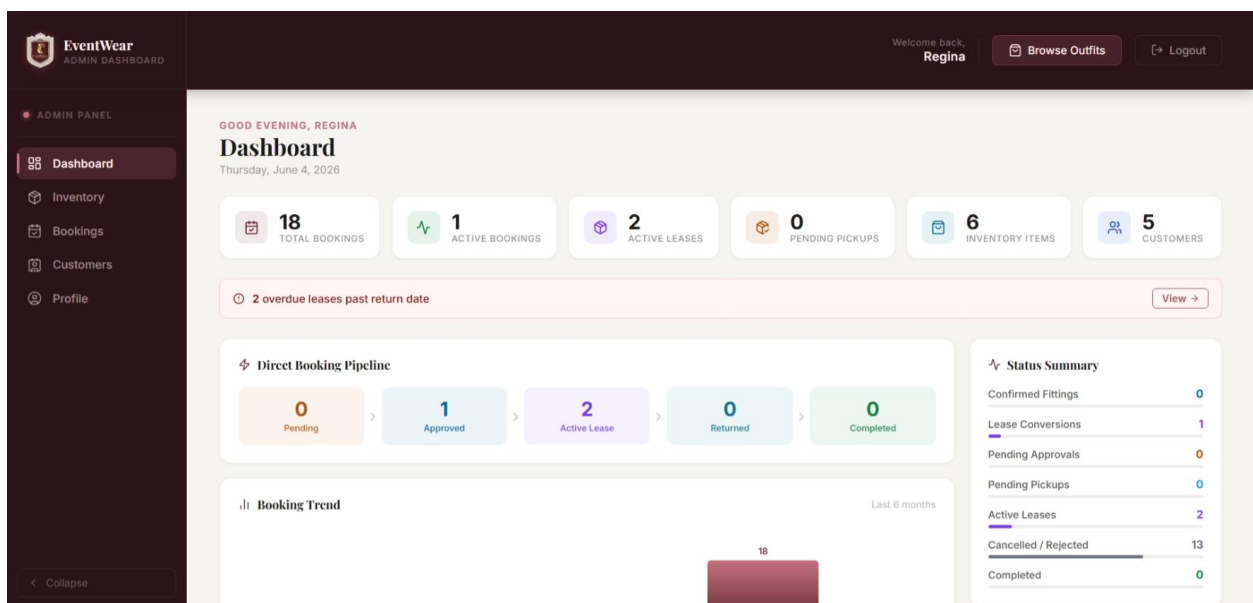
NEW PASSWORD

New Password

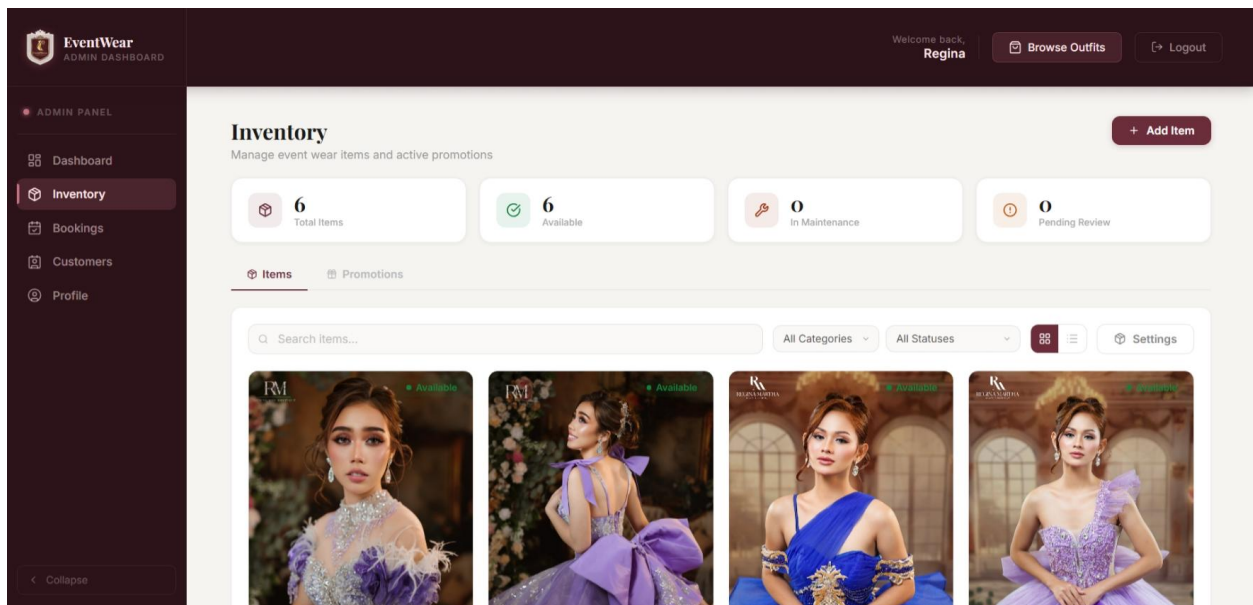
CONFIRM PASSWORD

Confirm Password

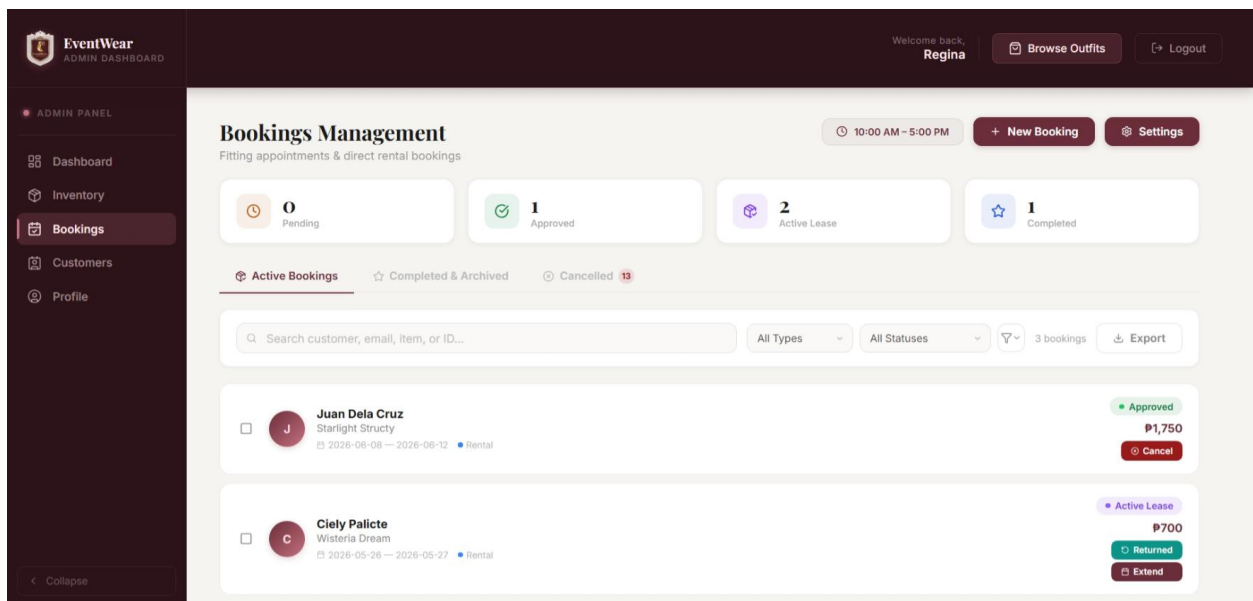
Admin Dashboard — Overview



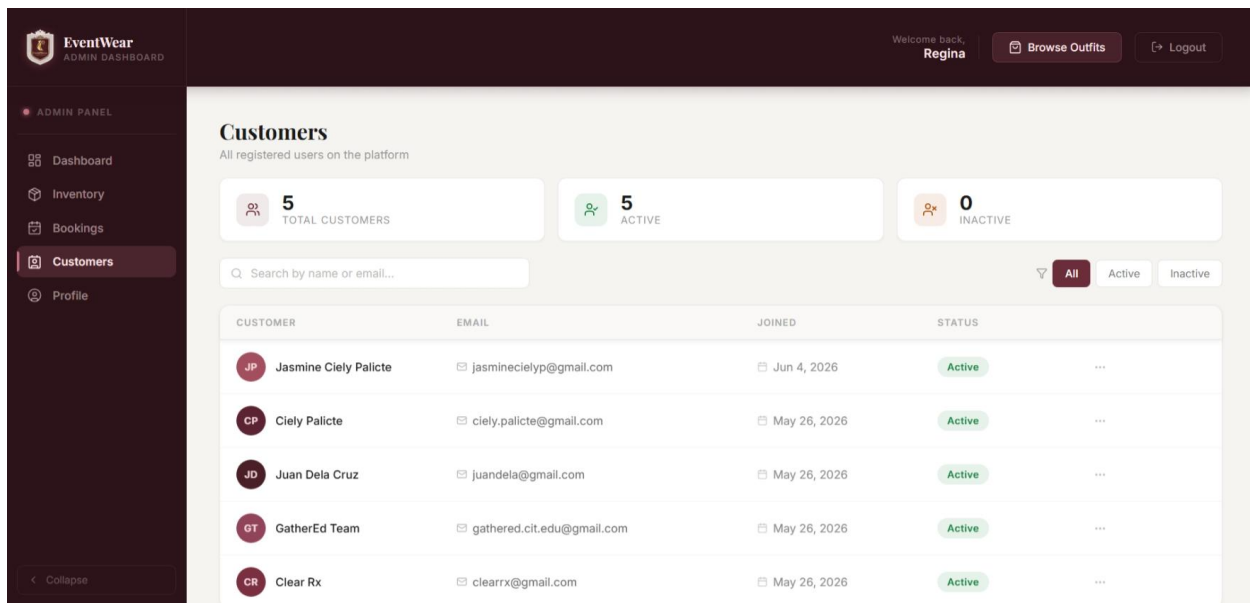
Admin Dashboard — Inventory



Admin Dashboard — Bookings



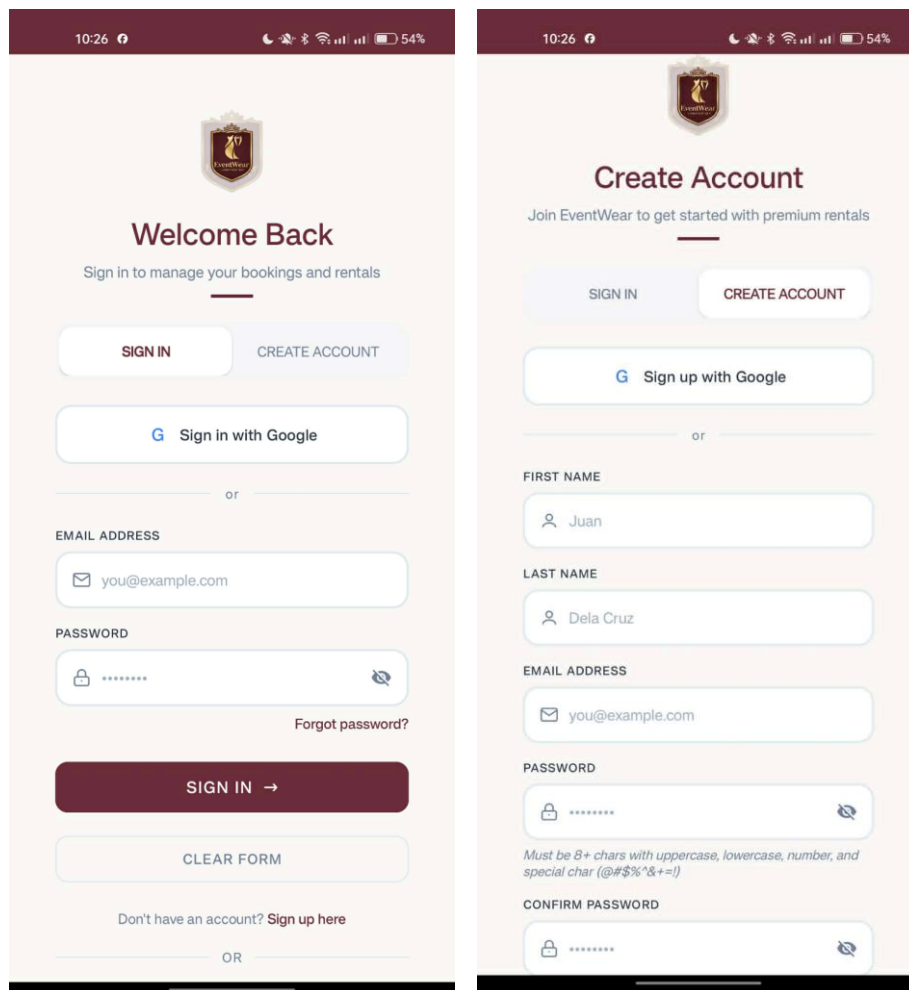
Admin Dashboard — Customers



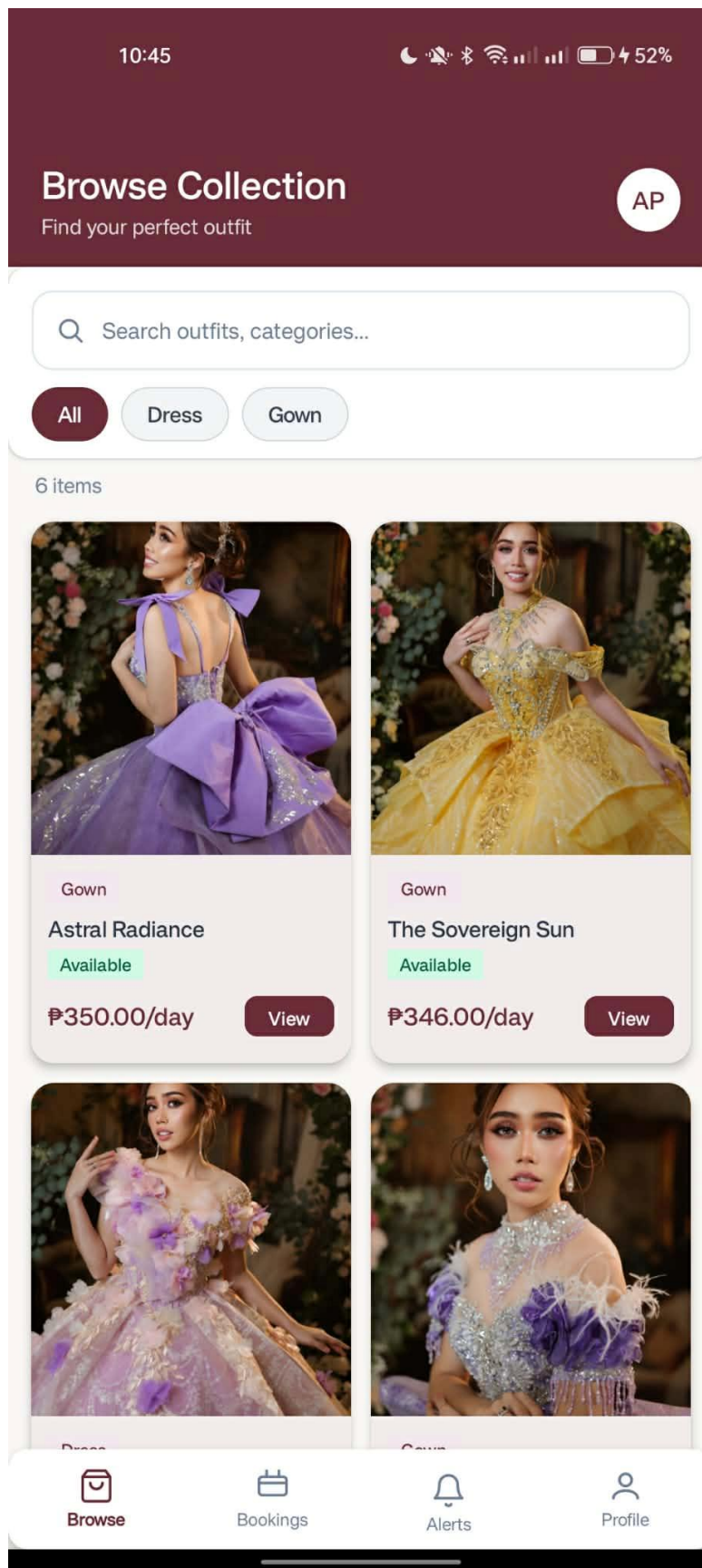
7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma

Login / Register Screen



Customer App — Browse Outfits Tab



Customer App — Book Fitting Dialog

10:46 52%

Browse Collection

Book Fitting

Astral Radiance

FITTING DATE

Tap to select date

TIME SLOT

Select date first

YOUR INFORMATION

FULL NAME

Althea Palicte

EMAIL

althea.palicte@gmail.com

PHONE

+63 9XX XXX XXXX

SPECIAL REQUESTS (OPTIONAL)

Any special requests or questions?

Confirm Fitting Booking

Browse Bookings Alerts Profile

Customer App — Book Direct Dialog

10:47 52%

Browse Collection

Book Direct Rental

Astral Radiance · ₱350.00/day

START DATE

Tap to select start date

END DATE

Select start date first

YOUR INFORMATION

FULL NAME

Althea Palicte

EMAIL

althea.palicte@gmail.com

PHONE

+63 9XX XXX XXXX

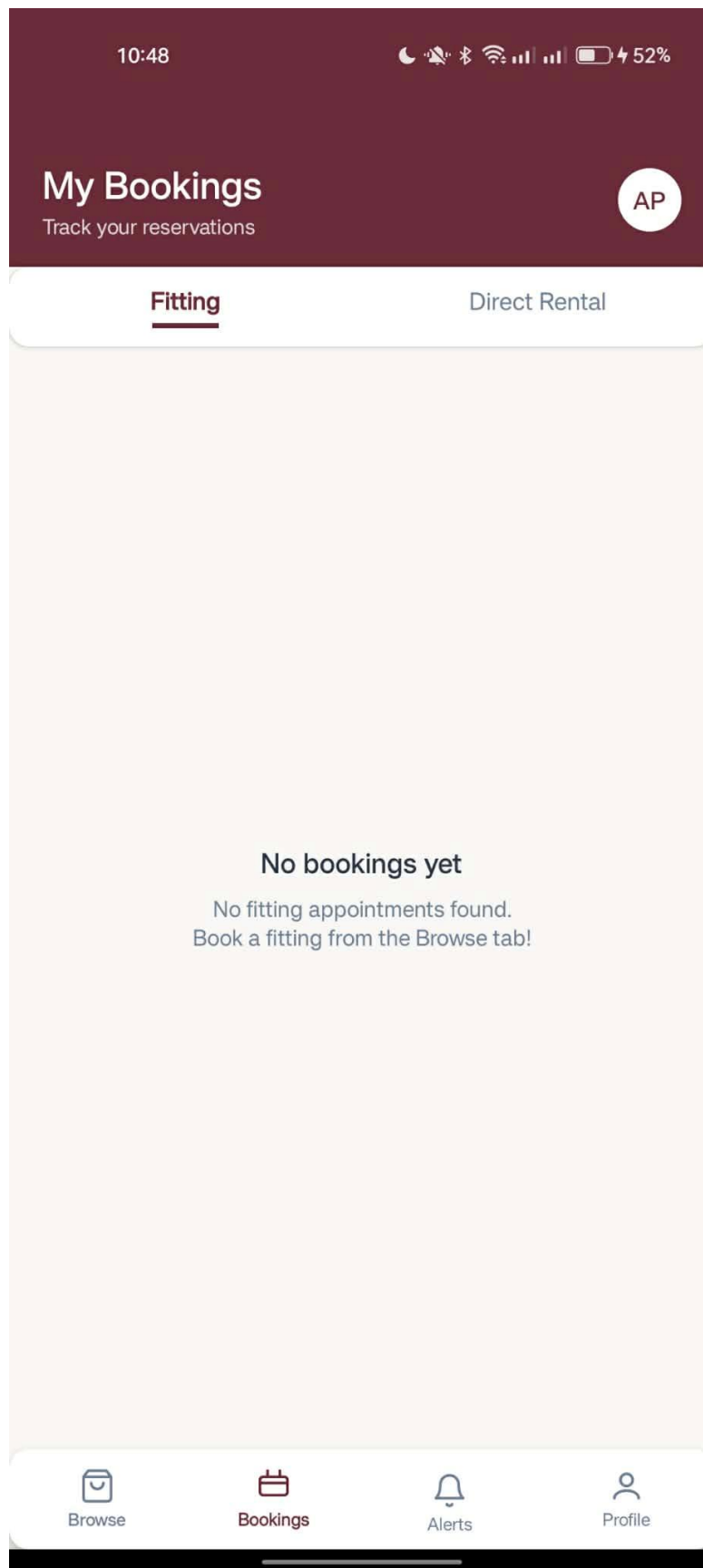
SPECIAL INSTRUCTIONS (OPTIONAL)

Any specific notes or requirements...

Confirm Rental Booking

Browse Bookings Alerts Profile

Customer App — My Bookings Tab



Customer App — My Profile Tab

10:49

 52%

My Profile

Manage your account

AP



Althea Palicte

althea.palicte@gmail.com

CUSTOMER

Personal Information

Edit

First Name

Althea

Last Name

Palicte

Email

althea.palicte@gmail.com

Phone

—

Account

CHANGE PASSWORD

Sign Out

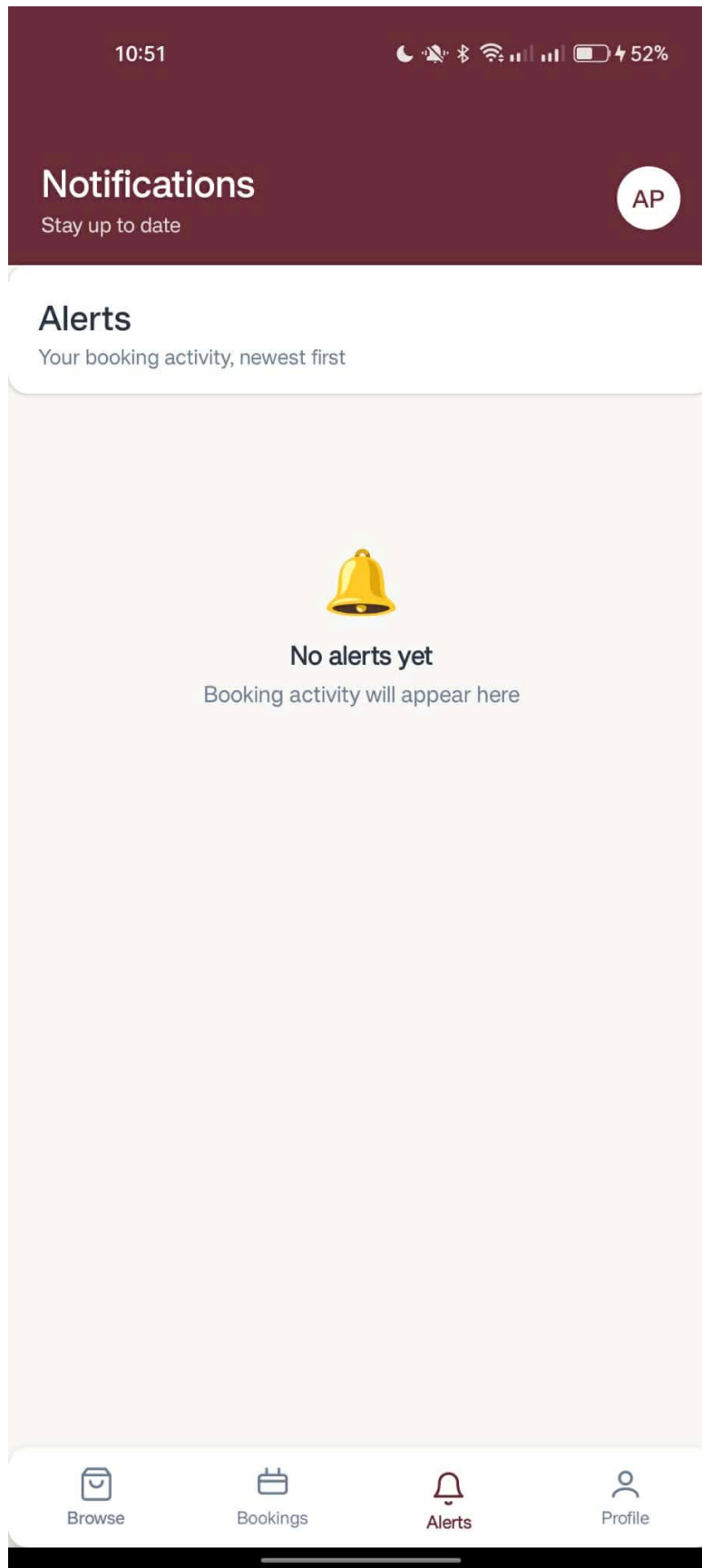
 Browse

 Bookings

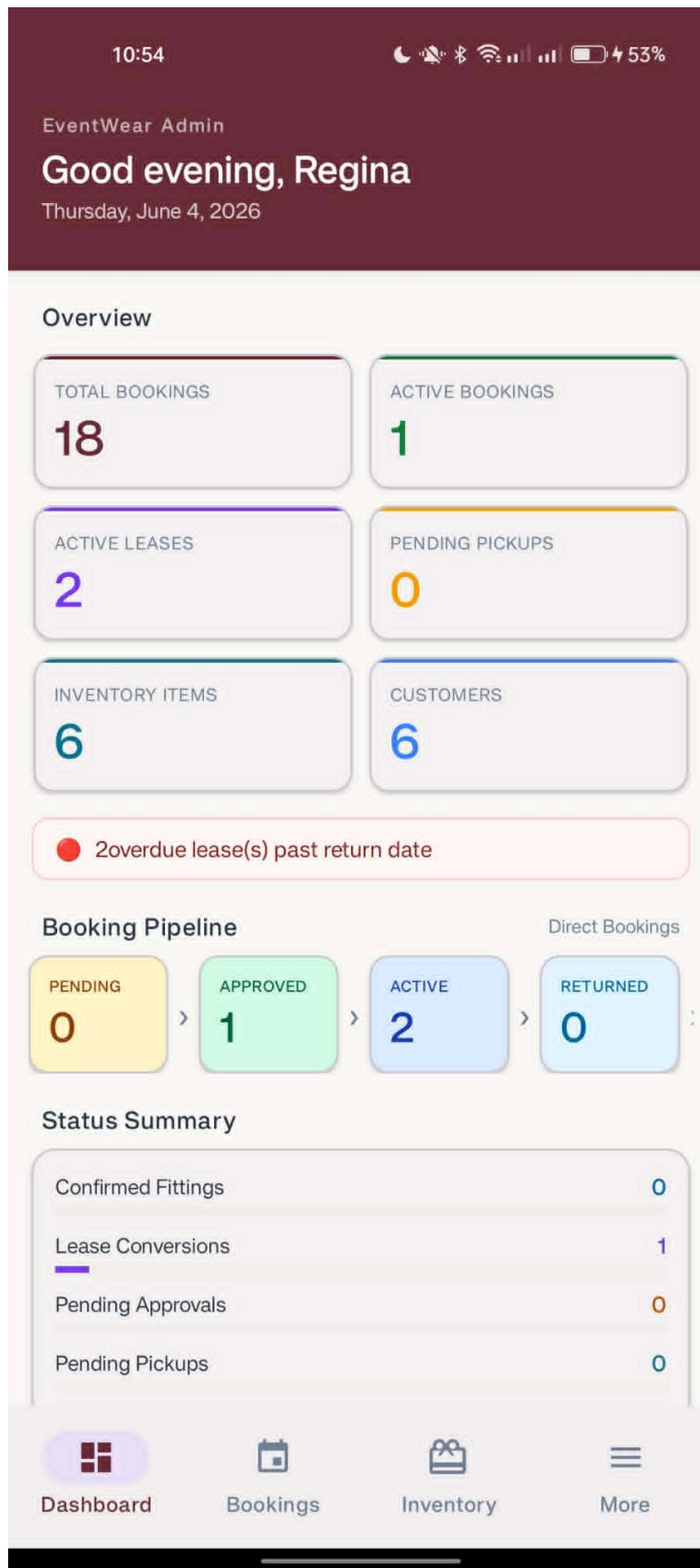
 Alerts

 Profile

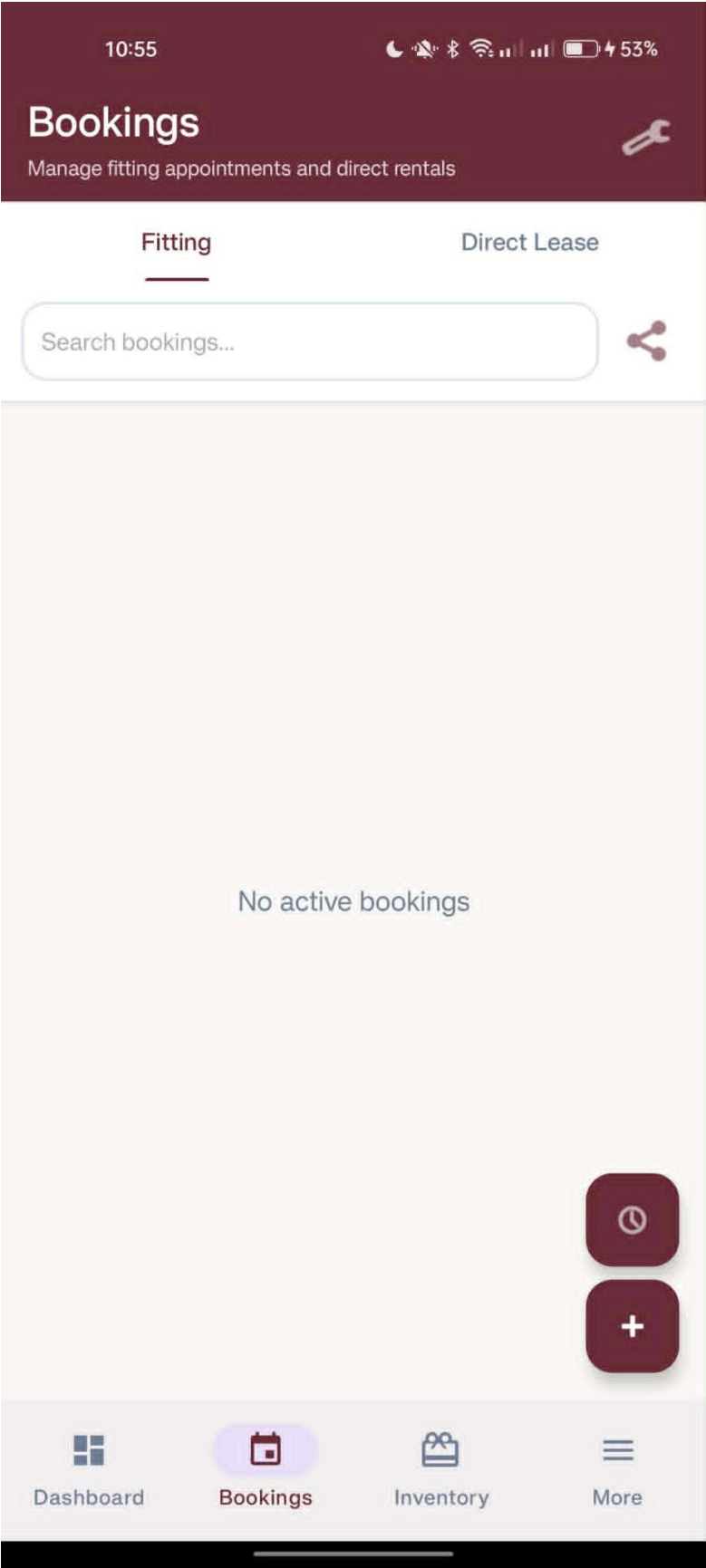
Customer App — Notification Tab



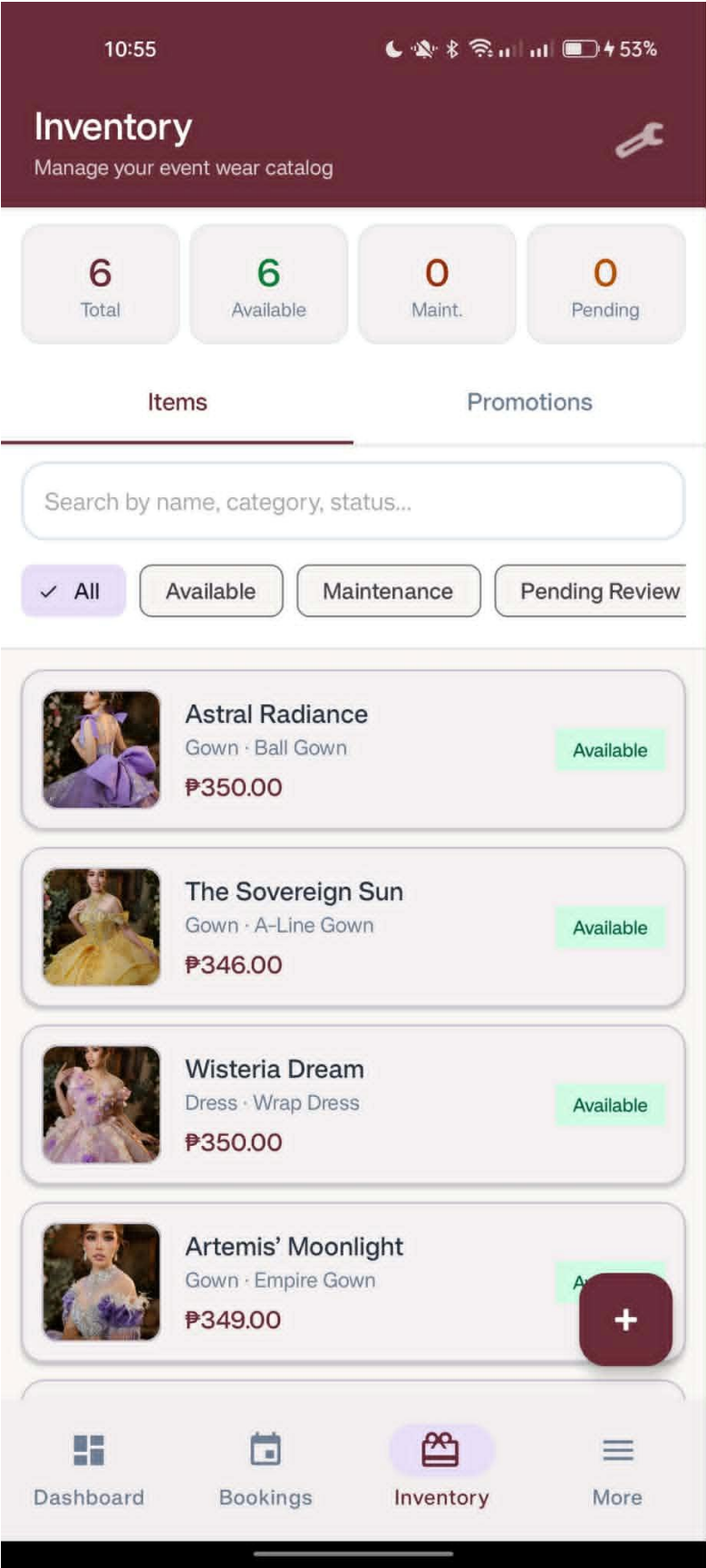
Admin App — Dashboard Tab



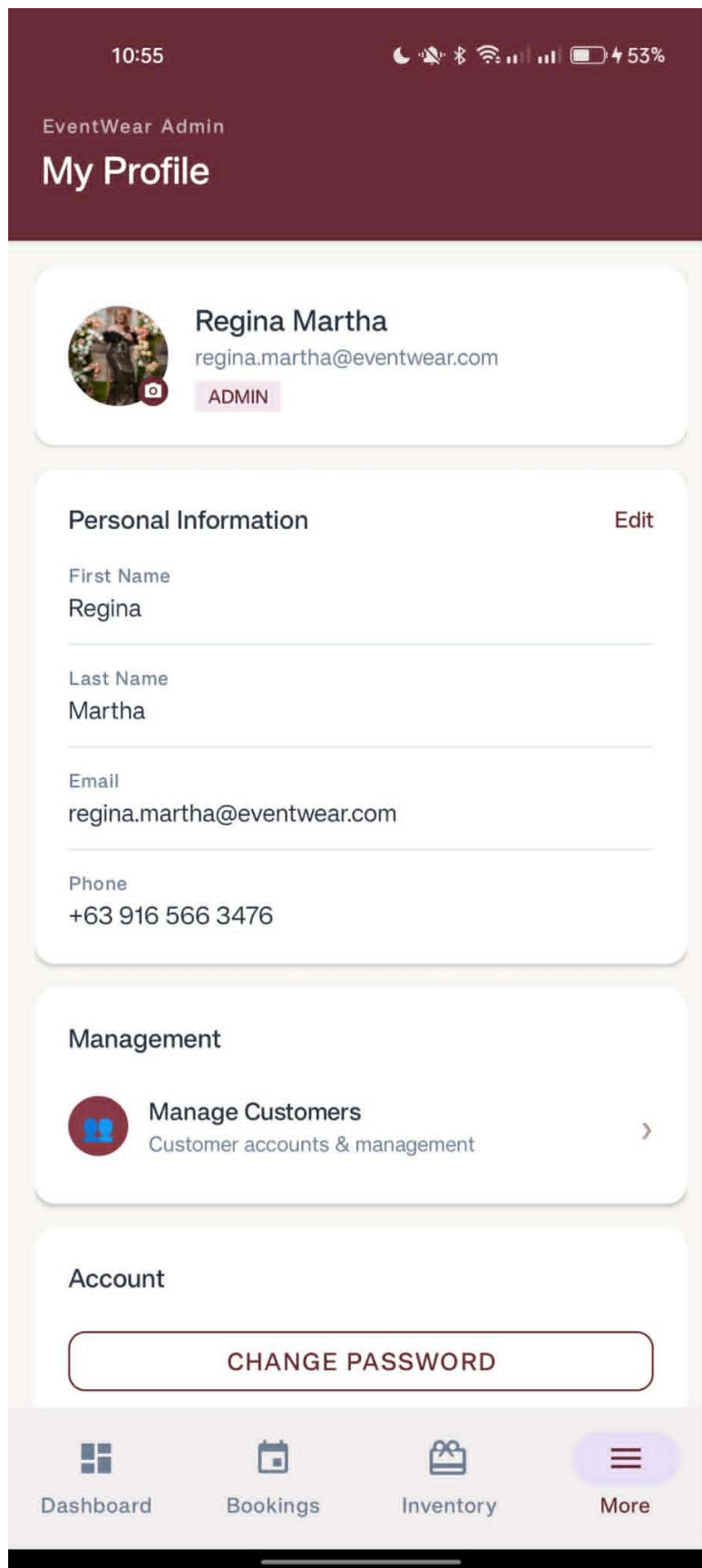
Admin App — Bookings Tab



Admin App — Inventory Tab



Admin App — Profile / More Tab



Design System:

Property	Value
Primary Color	Burgundy #6B2D39
Accent Color	Gold #D4A573
Background	Cream #F8F7F5
Success	Green #15803D
Error	Red #991B1B
Warning	Amber #B45309
Typography	System default — consistent sizing across web and mobile
Spacing	8px grid system
Touch Targets	Minimum 44×44px on mobile
Responsive Breakpoints	Mobile 360px+, Tablet 768px+, Desktop 1024px+
Web Navigation	Collapsible sidebar (admin and customer)
Mobile Navigation	Bottom navigation bar (4–5 tabs per role)
Status Badges	Named color tokens: status_pending, status_confirmed, status_active, status_completed, status_cancelled
Media	Item images and videos served from Supabase CDN; profile photos uploaded via multipart form

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

- Day 1–2: Project setup and initial documentation
- Day 3–4: Complete Functional Requirements Specification (FRS) and Non-Functional Requirements (NFR)

- Day 5–7: System architecture design — three-tier architecture (Spring Boot, React, Android)

Week 2: Detailed Design

- Day 1–2: API contract specification and endpoint design
- Day 3–4: Database schema design and ERD (9 tables: users, items, bookings, direct_bookings, promotions, password_reset_tokens, token_blacklist, bookingtime_settings, inventory_settings)
- Day 5–6: UI/UX wireframes for web (admin and customer dashboards) and mobile (admin and customer apps)
- Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3–4)

Week 3: Foundation

- Day 1: Spring Boot 4.0.3 project setup with Maven dependencies (Security, JPA, Validation, OAuth2, Mail, Actuator)
- Day 2: PostgreSQL database configuration, entity mapping (User, Item, Booking, DirectBooking, Promotion), and Supabase storage integration
- Day 3: JWT authentication implementation — token generation, validation, blacklisting on logout, token validation endpoint
- Day 4: Auth endpoints — register, login, logout, change password, forgot password (OTP via Gmail SMTP), verify OTP, reset password
- Day 5: Google OAuth2 social login integration and OAuth2 success/failure handlers

Week 4: Core Features

- Day 1: Inventory module — item CRUD, media upload to Supabase CDN, inventory settings (pricing, discounts), item status management
- Day 2: Promotions module — create, update, delete, per-item applicability; fitting booking module — slot availability check, working-day validation, slot capacity enforcement
- Day 3: Direct booking module — date availability check, lease pricing calculation (weekly/monthly discount), payment marking, lease extension and return

- Day 4: Booking lifecycle management — fitting-to-lease conversion, admin booking creation on behalf of customers, automated schedulers (FittingCancellationScheduler, LeaseActivationScheduler)
- Day 5: SSE service for real-time BOOKING_UPDATE and INVENTORY_UPDATE broadcasts; admin user management; global exception handler and input validation

Phase 3: Web Application (Week 5–6)

Week 5: Frontend Foundation

- Day 1: React 19 + Vite project setup, routing with React Router DOM, API client configuration with Axios, shared services (apiClient, sseService)
- Day 2: Public landing page — Hero section, How It Works (4-step process), Call to Action, Header and Footer components
- Day 3: Authentication pages — Login, Registration, Forgot Password (OTP flow), Google OAuth2 callback handler, session expiry detection
- Day 4: Customer dashboard layout — sidebar navigation, Dashboard overview (active/completed/cancelled booking summaries), Browse Outfits with grid/list toggle, search, and category filters
- Day 5: Customer booking flows — Book Fitting dialog (date picker, time slot selector, size, notes), Book Direct dialog (date range picker, price preview with discount calculation)

Week 6: Complete Web Features

- Day 1: Customer My Bookings — fitting and direct booking tabs, booking detail view, cancel and reschedule, edit direct booking dates
- Day 2: Customer profile management — view and edit profile, upload profile photo to Supabase, change password
- Day 3: Admin dashboard — booking pipeline metrics, inventory count, customer count, real-time refresh via SSE
- Day 4: Admin inventory management — item CRUD with image/video upload, status filter chips, promotions management, inventory settings configuration
- Day 5: Admin bookings management — fitting bookings (confirm, reschedule, fitting-to-lease, complete without lease), direct bookings (pickup, undo pickup, extend, return, mark payment), booking settings; admin customer management (search, filter, activate/deactivate)

Phase 4: Mobile Application (Week 7–8)

Week 7: Android Foundation

- Day 1: Android Studio project setup — Kotlin, Gradle KTS, Retrofit2, OkHttp3, Gson, Coroutines, ViewBinding, Material 3; brand theme setup (Burgundy, Gold, Cream)
- Day 2: Shared infrastructure — ApiClient singleton (adminApi, customerApi), SessionManager, Models.kt (all data classes), authentication screens (Login, Registration)
- Day 3: Auth routing — role-based navigation: ADMIN → AdminDashboardActivity, CUSTOMER → DashboardActivity; SseClient with auto-reconnect
- Day 4: Admin mobile app — AdminDashboardActivity with 5-tab BottomNavigationView; Dashboard fragment (5-stage direct booking pipeline + fitting snapshot)
- Day 5: Admin fragments — Bookings (FittingBookings, DirectBookings with status update), Inventory (item list with status filter chips and search), Customers (paginated list with detail view)

Week 8: Complete Mobile App

- Day 1: Admin Promotions fragment — multi-select item applicability, create/edit/delete; Admin Settings and Profile fragments
- Day 2: Customer DashboardActivity — 4-tab BottomNavigationView (Dashboard, Browse Outfits, My Bookings, My Profile); CustomerBrowseFragment with 2-column grid, search, and category chips
- Day 3: Customer booking dialogs — ItemDetailBottomSheet, BookFittingDialog (CalendarPickerDialog + TimeSlotPickerDialog), BookDirectDialog (date range picker, price preview with discount logic)
- Day 4: Customer My Bookings — fitting and direct booking adapters with cancel support; booking detail bottom sheets; CustomerProfileFragment with edit profile, change password, and logout
- Day 5: UI polish, edge case handling, APK build, and testing on physical device and emulator

Phase 5: Integration & Deployment (Week 9–10)

Week 9: Integration Testing

- Day 1: End-to-end testing across all three platforms — web, Android admin, and Android customer — verifying booking flows, real-time SSE updates, and role-based access
- Day 2: Bug fixes — cross-platform inconsistencies, slot availability edge cases, date overlap validation, SSE reconnect behavior
- Day 3: Security review — JWT blacklisting verification, CORS policy validation, role-based endpoint enforcement, input validation completeness
- Day 4: Performance testing — API response times, Supabase CDN media load times, SSE connection stability
- Day 5: Documentation updates — API contract finalization, ERD verification, SDD completion

Week 10: Deployment

- Day 1: Backend deployment — Spring Boot JAR to cloud hosting; environment variable configuration (database URL, JWT secret, Supabase keys, Gmail SMTP credentials)
- Day 2: Web application build and deployment — Vite production build, static hosting, BASE_URL update for production API
- Day 3: Mobile APK generation — release build, BASE_URL update in ApiClient.kt to production server, APK signing and distribution
- Day 4: Final end-to-end testing on production — verify all API calls, OAuth2 redirect URIs, Supabase CDN URLs, and SSE connectivity
- Day 5: Project submission — final documentation package, source code, and demo preparation

Milestones:

Milestone	Target	Description
M1	End of Week 2	All design documents complete — FRS, NFR, ERD, API contract, architecture diagram, wireframes
M2	End of Week 4	Backend API fully functional — all endpoints implemented, tested, and secured
M3	End of Week 6	Web application complete — admin and customer dashboards fully integrated with the backend

M4	End of Week 8	Mobile application complete — Android admin and customer apps functional with full API integration
M5	End of Week 10	Full system deployed and integrated — all platforms live, tested, and submitted

Critical Path:

1. JWT authentication and Google OAuth2 (Week 3) — blocks all protected endpoints and role-based routing
2. Inventory item and media management (Week 4) — blocks all booking flows on web and mobile
3. Fitting and direct booking APIs (Week 4) — blocks all customer-facing booking features
4. SSE service implementation (Week 4) — blocks real-time dashboard updates on all platforms
5. Cross-platform integration testing (Week 9) — final gate before deployment

Risk Mitigation:

- Implement the simplest working version of each feature first before adding edge case handling
- Test API integration points early — connect web and mobile to the backend as each endpoint is completed rather than waiting until the end
- Maintain working Git checkpoints at each phase milestone using feature branches
- Prioritize core booking flows (fitting booking, direct booking, booking lifecycle) before secondary features (promotions, settings, SSE)
- Use a local IP-based BASE_URL in development and switch to the deployed URL only during the final deployment phase
- Keep all credentials (Supabase keys, JWT secret, Gmail credentials) in environment variables — never hardcoded in source files